

Cloud Job Orchestrator

Sistema **event-drive** per l'orchestrazione
resiliente di job **asincroni** in Azure



Contesto e Problema Affrontato

Problema

Il progetto nasce per risolvere un problema preciso:

Gestire job asincroni senza bloccare il client, disaccoppiando interfaccia, API e fase di processing.

Obiettivo

L'obiettivo quindi non è costruire una semplice applicazione in grado di eseguire job generici, ma una **pipeline resiliente** in cui accettazione della richiesta, esecuzione e monitoraggio dello stato siano **separati**.

Il sistema deve **assorbire i picchi**, **gestire retry**, supportare **aggiornamenti di stato affidabili** e mantenere il tracciamento del job.

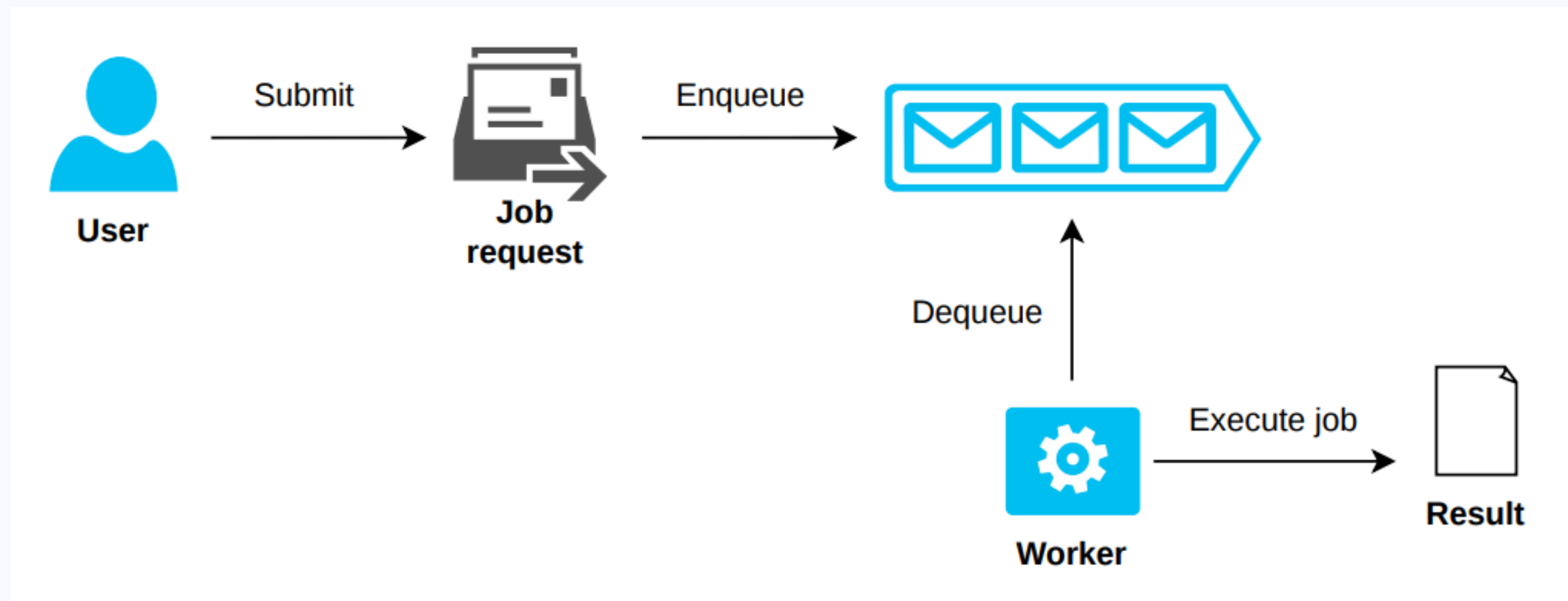


Alcune terminologie utilizzate

Job: Operazione asincrona da eseguire

Worker: Processo che si occuperà di eseguire il job

Queue: Coda usata per assorbire i picchi di richieste e gestire i rery





Requisiti principali

1. Il Sistema permette all'utente di:
 - **Autenticarsi** in modo sicuro
 - **Creare un job** asincrono
 - **Monitorare** l'esecuzione dei worker attivi
 - **Ricevere aggiornamenti** di stato in real-time
 - **Scaricare l'output** finale

2. Il Sistema assicura:
 - **Invio** del job **non bloccante** lato client
 - Comportamento **at-least-once** con **retry** del messaggio
 - **Scalabilità orizzontale** dei worker
 - **Assorbimento dei picchi** tramite buffering su coda



Caso d'uso scelto

Elaborazione asincrona di immagini

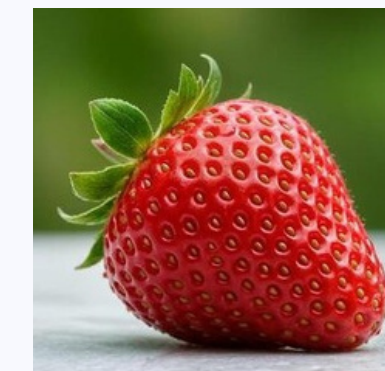
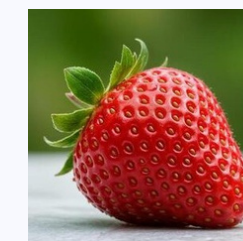
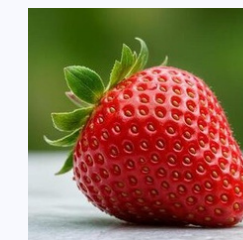
I caso d'uso implementato è l'orchestrazione di job relativi all'**elaborazione di immagini**, scelti come casi d'uso rappresentativi.

Motivazione della scelta

- Si tratta di **operazioni comuni** in applicazioni reali
- Richiedono **tempi** di esecuzione **non trascurabili**
- Si adattano naturalmente a un modello asincrono, come CJO

Descrizione dei job types

- **Background Removal:** Rimozione automatica dello sfondo con isolamento del soggetto principale
- **Image Upscale:** Aumento della risoluzione dell'immagine di 2× mantenendo la qualità percepita





Azure Services Utilizzati



Azure Functions

- **Ruolo:** API HTTP, Worker queue-trigger, Timer di cleanup dei dati
- **Motivo:** Modello **Serverless**, Separazione naturale dei trigger e **scaling gestito**



Azure Service Bus

- **Ruolo:** coda job q-jobs.
- **Perché:** disaccoppia intake da execution, bufferizza picchi, abilita retry dei worker



Azure Cosmos DB

- **Ruolo:** stato e metadati job.
- **Perché:** datastore operativo a bassa latenza, usato come **source of truth**.



Azure Blob Storage

- **Ruolo:** persistenza input/output e download via SAS temporaneo.
- **Perché:** storage adatto a payload binari, separato dai metadati.



Azure Services Utilizzati



Azure App Service / Web App

- **Ruolo:** Hosting frontend Express.
- **Motivo:** Hosting semplice della UI con configurazione via app settings.



Azure SignalR Service

- **Ruolo:** Push realtime eventi job a utenti specifici.
- **Perché:** aggiornamenti real-time senza polling continuo, con fallback quando necessario.



Microsoft Entra ID

- **Ruolo:** Identity provider per login e protezione API via bearer token.
- **Perché:** Gestione centralizzata identità in contesto Azure.



Application Insights

- **Ruolo:** Telemetria e osservabilità runtime.
- **Perché:** Analisi concorrenza worker, failure e performance

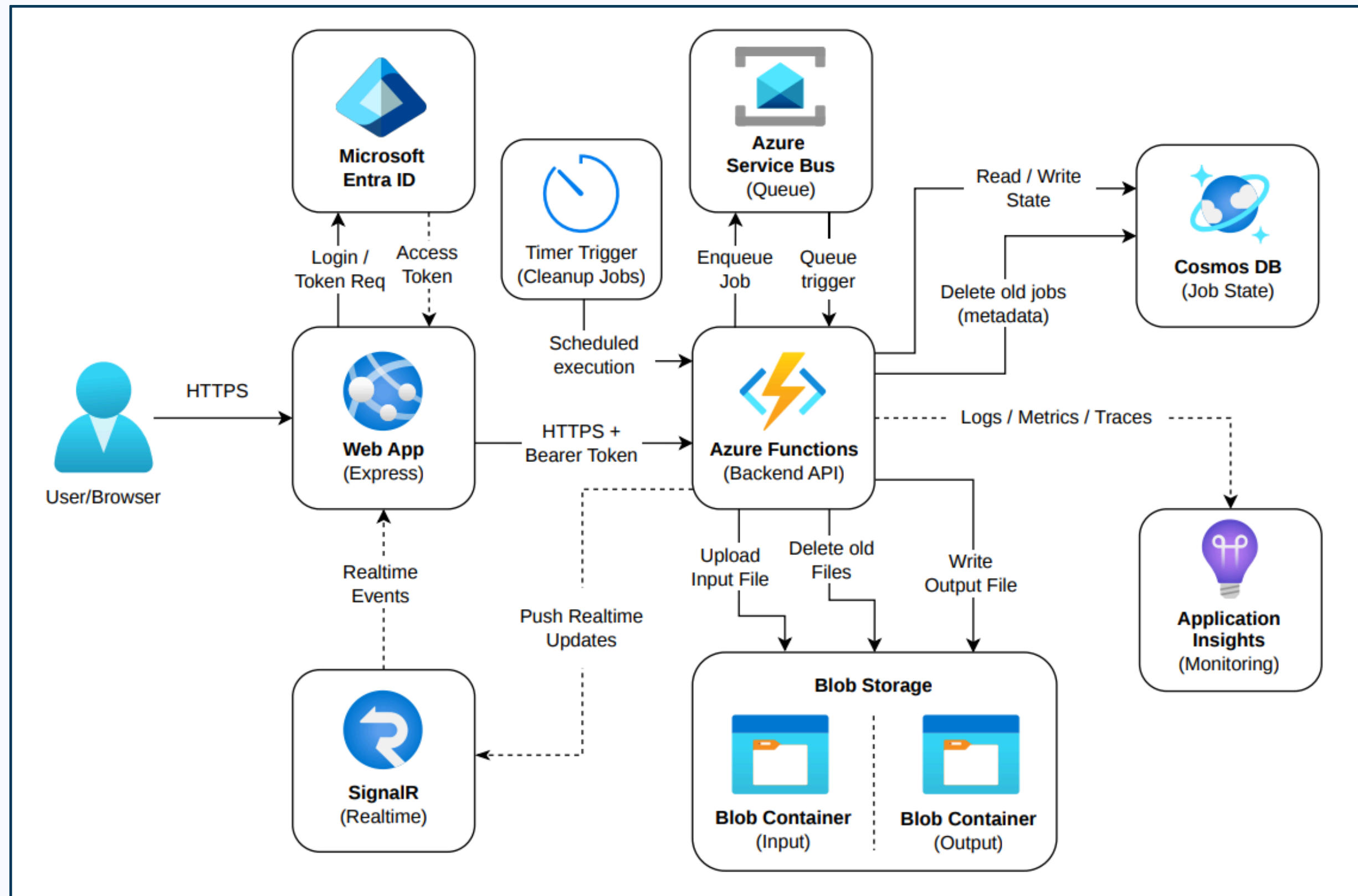


Workflow End-To-End

1. L'utente **effettua il login** e **crea un job** dalla UI.
2. L'API **valida token**, tipo di job e input.
3. Il file viene caricato su **Blob Storage**.
4. In **Cosmos DB** viene creato il record iniziale con stato `queued`.
5. L'API invia un messaggio leggero su **Service Bus**.
6. Il worker **consuma il messaggio**, recupera il file, esegue il processing e salva l'output.
7. Cosmos **viene aggiornato** con stato, progresso ed eventuali errori.
8. **SignalR** invia gli eventi realtime al client.
9. Se il realtime non è disponibile, il frontend usa il **polling come fallback**.
10. Quando il job è concluso, l'utente **scarica l'output** tramite link **SAS temporaneo**.



Architettura





Azure Functions

JobsCreateApi

Valida la richiesta, **carica l'immagine** su Blob, **crea il documento** job in Cosmos e **pubblica il messaggio** su Service Bus.

JobsGetStatusApi

Legge lo stato del job da Cosmos e funge da **fallback** se il realtime SignalR non è disponibile.

JobsGetOutputLinkApi

Genera un **link SAS** temporaneo per scaricare l'output dal Blob.

RealtimeNegotiateApi

Negozia la connessione realtime con il client.

JobsWorker

Trigger su Service Bus che **esegue il processamento asincrono**.

CleanupOldCompletedJobs

Eseguita **ogni 15 minuti**; **rimuove i job completati** da più di un'ora e i blob associati, ma conserva i failed per analisi e monitoraggio.



Motivazioni dell'utilizzo

- Service Bus **separa intake ed execution**: la API accoda → il worker consuma.
- Questo permette di **assorbire i picchi** e di **scalare orizzontalmente i worker** in modo indipendente.
- Il messaggio viene ricevuto in modalità **peek-lock**: una volta acquisito, non è visibile agli altri consumer finché non viene completato o rilasciato (a causa di un'eccezione).
- Se il worker fallisce, il messaggio torna disponibile secondo la logica di retry della coda; quando supera il limite configurato (**MaxDeliveryCount**), viene spostato in **dead-letter** queue.



Cosmos DB

È la **source of truth** del job:
contiene un **documento JSON**
con: stato, progress, tentativi,
timestamp, riferimenti ai file ed
eventuale errore.



```
{
  "id": "d7f76fa0-9c17-4e7e-9b6f-317a7fec6dec",
  "pk": "03a938d9-ea6a-4b32-a409-e7b4a61acbf1",
  "type": "background_removal",
  "status": "failed",
  "progress": 0.1,
  "attempts": 3,
  "createdAt": "2026-03-28T16:57:33.739276+00:00",
  "updatedAt": "2026-03-28T16:57:56.904401+00:00",
  "correlationId": "88f88350-4c4a-4966-922b-3b69c25a4bde",
  "inputRef": {
    "container": "input",
    "blobName": "03a938d9-ea6a-4b32-a409-e7b4a61acbf1/d7f76fa0-9"
  },
  "parameters": {
    "fail": true
  },
  "_rid": "OMRMALfmY1UEAgAAAAAAAA==",
  "_self": "dbs/OMRMAA==/colls/OMRMALfmY1U=/docs/OMRMALfmY1UEAgAAA",
  "_etag": "\"0a00752b-0000-1600-0000-69c808940000\"",
  "_attachments": "attachments/",
  "error": {
    "code": "DEMO_FORCED_FAILURE",
    "message": "Forced demo failure (parameters.fail=true)",
    "stage": "forced-demo-failure"
  }
}
```

Blob Storage

- conserva i binari separando **input** e **output** in container distinti.
- Il download non espone direttamente lo storage: il backend genera un **SAS temporaneo** con scadenza breve.



 input

 output

Storage



Scelte progettuali lato codice

Autenticazione e Configurazione delle API

- Il login con **MSAL** serve ad accedere all'interfaccia, ma non basta a sbloccare le API.
- Ogni richiesta verso il backend invia un access token nel header "Authorization: Bearer ..."
- Nel backend, ogni endpoint richiama `get_user_from_request(req)`, che valida il token controlla le chiavi pubbliche **JWKS** del tenant, controlla aud e iss, e ricava oid e tid.
- Le API restano quindi protette anche dopo il login, perché la validazione **viene ripetuta ad ogni chiamata**.

Assenza di Hard-Coding

- I parametri di ambiente non sono **hard-coded**: nomi risorse, chiavi e configurazioni vengono letti da **variabili d'ambiente**, con esempi in `.env.example`, `local.settings.example.json`



Scelte progettuali lato codice

Worker asincrono e standardizzazione del processo

- Nel codice il dispatch dei job è centralizzato in `_process(job_type, input)`, quindi in base al tipo di job richiama la funzione dedicata.
- Oggi supporta **background_removal** e **image_upscale**; per aggiungere un nuovo job basta introdurre una nuova funzione e un nuovo ramo nel dispatcher.
- Questo riduce il coupling tra worker e UI e rende più semplice l'estensione del sistema.

Nel file host.json il trigger Service Bus usa `maxConcurrentCalls: 2`. La scelta è stata mantenuta dopo test e osservazioni con Application Insights per limitare la saturazione di RAM sulle singole istanze e ridurre crash sotto carico.



Limiti e Sviluppi Futuri

- Estensione del sistema con **nuovi tipi di job**, sfruttando l'architettura modulare del worker.
- Supporto a **input multipli** con validazioni più avanzate e flessibili
- **Osservabilità avanzata** con metriche di business e tracing distribuito
- **Persistenza storica dei job** per consultazione a lungo termine (frontend o datastore dedicato)
- Evoluzione verso una **piattaforma operativa più completa**, oltre il semplice processing di job veloci



UI Screenshot

1) Form di creazione del job

Cloud Job Orchestrator — Demo UI

Bentornato: User_1

Upload an image and start a processing job. The system will handle execution, track progress in real time, and provide the output file when ready.

Job Type
background_removal

Image
Scegli file Nessun file selezionato

Create Job

☐ Force DEMO failure

Logout

2) Job Execution Card

Job ID: 9ebae2e7-4d27-49b0-9250-666b862e5dc5

Filename
input.jpg

Job Type
background_removal

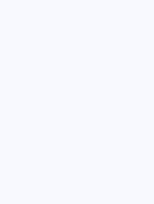
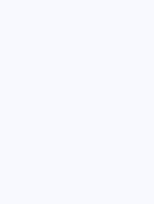
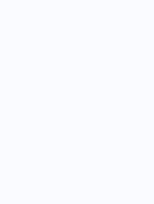
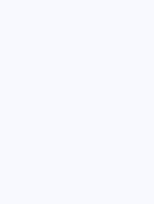
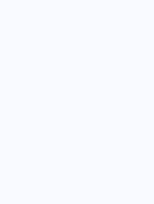
Status
DONE

Progress

Download Output

```
[22:47:28] Job completed successfully
[22:47:28] Uploading output to Blob Storage
[22:47:12] Transformation still running: background_removal
[22:46:52] Transformation still running: background_removal
[22:46:32] Applying transformation: background_removal
[22:46:31] Processing started | attempt 1 | deliveryCount=1 | type=background_removal
```

Demo accessibile al link: <https://cjo-frontend-bsbqc4d5bdf3eec7.switzerlandnorth-01.azurewebsites.net>



Grazie per l'attenzione <3